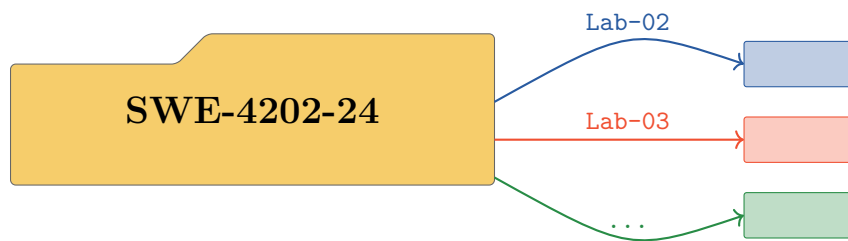


A Student's Handbook to Git, GitHub, and Your Course Folder

A practical, picture-first guide for SWE-4202



Course: Object Oriented Concepts I

Designed so that you never have to ask:

“Why is my project not opening?” or “Why did my push fail?” again.

Contents

1	Why This Handbook Exists	2
2	What Is a Git Repository, Really?	2
3	Your Course Folder Structure	3
4	Opening the Right Folder in IntelliJ IDEA	3
4.1	Step-by-step: opening RideNow correctly	4
5	The Three Areas of Git	4
6	The Git Commands You Will Actually Use	5
6.1	<code>git clone</code> — get the course repository onto your machine	5
6.2	<code>git branch</code> — your personal workspace	5
6.3	<code>git add</code> and <code>git commit</code> — save a snapshot	6
6.4	<code>git push</code> — send your commits to GitHub	6
6.5	<code>git pull</code> — get the latest from GitHub	6
6.6	<code>git merge</code> — combine two branches	7
7	The Workflow for This Course	8
7.1	The day-to-day rhythm	8
8	Common Mistakes and How to Avoid Them	9
9	Quick Reference Card	10
10	Closing Words	10

1. Why This Handbook Exists

You are taking a course where every lab assignment is delivered through a single Git repository. Your instructor pushes new labs to GitHub. You pull them down, do your work on your own branch, and push your solutions back. This sounds simple, but most beginners get tripped up by two things:

1. **They don't understand what a Git repository *is* as a folder on disk.** They treat it as a magical thing rather than a regular directory with a hidden `.git` subfolder.
2. **They open the wrong folder in their IDE.** They open the entire course folder (SWE-4202-24) or a lab folder (Lab-03) in IntelliJ IDEA, instead of the actual *project* folder (Lab-03/RideNow).

This handbook fixes both. By the end you will know:

- What a Git repository folder really is.
- Why the course is structured the way it is.
- Which folder to open in IntelliJ IDEA — and why.
- What `clone`, `branch`, `commit`, `pull`, `push`, and `merge` actually do.
- How your branch, your classmates' branches, and the teacher's `master` branch live together on GitHub.

2. What Is a Git Repository, Really?

A **Git repository** is just a folder on your computer that has a hidden subfolder called `.git` inside it. That `.git` folder is the entire history of every file you have ever committed — every change, every branch, every author, every date. If you delete `.git`, the folder becomes a normal folder again and Git forgets everything.

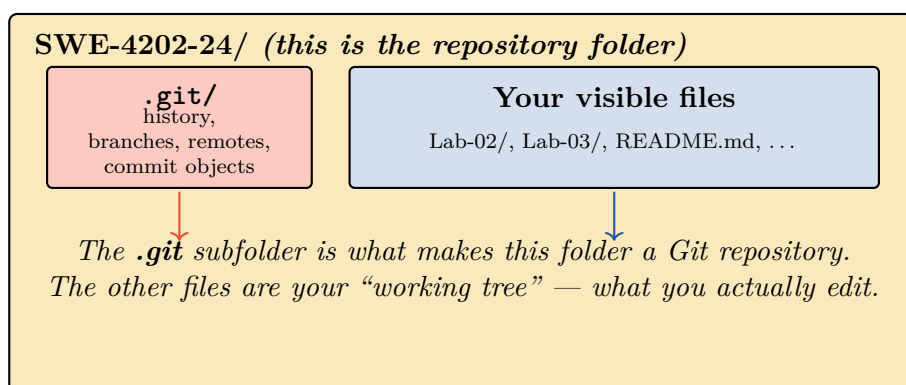


Figure 1: A Git repository folder = your working files + a hidden `.git` folder.

Note

On Linux/macOS the `.git` folder is hidden because its name starts with a dot. On Windows you may need to enable "Show hidden files" in Explorer to see it. **Do not edit anything inside `.git` by hand.** Git takes care of it.

3. Your Course Folder Structure

The repository for this course is named SWE-4202-24. Inside it, every lab gets its own folder. Some labs contain a *single project* (like Lab-03/RideNow); other labs may contain *multiple projects* (one per task or per problem). The structure looks like this:

```

SWE-4202-24/          <-- the Git repository (
  contains .git/)      (hidden -- Git's internal
|-- .git/              database)
|-- README.md
|-- Lab-02/
|   |-- README.md
|   '-- (future projects, e.g. ProjectA/, ProjectB/)
|-- Lab-03/
|   '-- RideNow/      <-- OPEN THIS in IntelliJ
      IDEA
|       |-- .gitignore
|       |-- RideNow.iml
|       |-- src/
|       |   |-- Driver.java
|       |   |-- Location.java
|       |   |-- Passenger.java
|       |   '-- Trip.java
|       |-- test/
|       |-- out/
|       '-- .idea/
'-- Lab-04/           (later in the course)

```

Figure 2: The course folder layout. Each *project* (e.g. RideNow) sits inside a lab folder, and the whole thing is a single Git repository.

Three nesting levels — and only one of them is a Git repository:

1. SWE-4202-24/ — the **repository**. It contains the `.git` folder.
2. Lab-03/ — a **lab folder**. Just a normal subfolder. It groups related projects.
3. Lab-03/RideNow/ — a **project**. This is what IntelliJ IDEA understands as a project (it has `src/`, `.idea/`, an `.iml` file, etc.).

4. Opening the Right Folder in IntelliJ IDEA

This is the single most common source of confusion. Let's settle it with a picture.

Do This

Rule of thumb: Open the folder that contains `src/` and `.idea/`. For Lab-03, that is Lab-03/RideNow. For a future lab with multiple projects, you would open each project separately (or use “Open as a new window”).

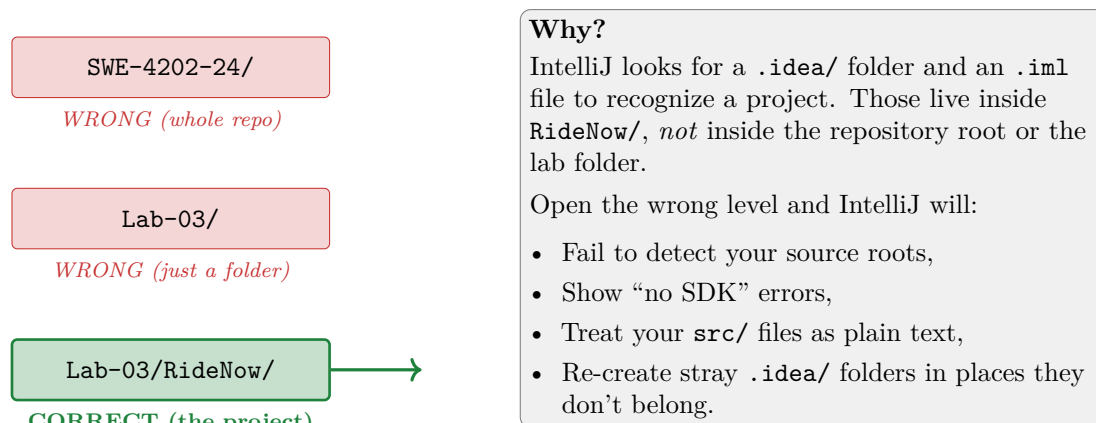


Figure 3: Three folders, only one is the IntelliJ project.

Common Mistake

If you have already mistakenly opened the wrong folder, IntelliJ may have created an unwanted `.idea/` directory at that level. Close the project, delete the stray `.idea/` (only the one in the wrong location!), and reopen the correct project folder.

4.1. Step-by-step: opening RideNow correctly

1. Launch IntelliJ IDEA.
2. Click **File** → **Open...** (or “Open” on the welcome screen).
3. Navigate to: `SWE-4202-24/Lab-03/RideNow`.
4. Select the `RideNow` folder itself (**do not open it and pick a file inside**).
5. Click **OK**. Choose “This Window” or “New Window” as you prefer.

5. The Three Areas of Git

Before we look at commands, you need to know that every file in a Git repository lives in one of three “areas”. Almost every Git command moves files between these areas.



Figure 4: The three areas of Git. Files flow from where you edit them, to a staging area, into the permanent history.

Note

A **commit** is a snapshot of the staging area, with a message attached. Commits are what GitHub stores, and what you can travel back to later.

6. The Git Commands You Will Actually Use

Every command below is something you will run in a terminal opened *inside the repository folder* (or via IntelliJ's built-in Git tools). The order in this section roughly matches the order you will need them in.

6.1. `git clone` — get the course repository onto your machine

Command: `git clone`

Purpose: Download a full copy of a remote repository (and all its history) to your computer.

When to use: *Once*, at the very start of the course.

Example:

```
git clone https://github.com/<instructor>/SWE-4202-24.git
cd SWE-4202-24
```

After this, you have the entire course folder shown in Figure 2, with `.git` already inside it.

Common Mistake

Do **not** run `git clone` again every time a new lab is released. You only clone once. To get new labs later, you use `git pull` (Section 6.5).

6.2. `git branch` — your personal workspace

A **branch** is an independent line of development. You and your classmates can each have your own branch and edit the same project without stepping on each other's toes. The teacher keeps the *official* version on a branch called `master` (or `main`).

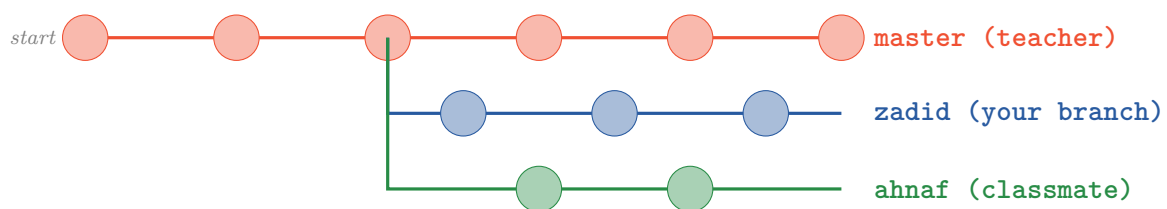


Figure 5: The teacher's `master` branch is the trunk. Each student branches off and makes their own commits.

Command: `git branch` / `git switch`

Purpose: Create or switch between branches.

Typical first-time-on-a-new-machine flow:

```
git branch # see which branches exist locally
```

```
git branch <your-name>    # create a branch named after you
git switch <your-name>    # move onto that branch
```

A shortcut for “create and switch” is:

```
git switch -c <your-name>
```

Do This

Use a branch name that identifies you clearly — for example your university ID or your first name in lowercase, like **zadid** or **210042xxx**. Your instructor will tell you the exact convention.

6.3. git add and git commit — save a snapshot

You finished a piece of work? Save a snapshot. That snapshot is called a **commit**.

Command: git add + git commit

Purpose: Record a snapshot of your work in the repository’s history.

Two-step process:

```
git status                # see what changed
git add Lab-03/RideNow/src/Driver.java # stage one file
git add .                 # OR stage everything
                           changed
git commit -m "Implement Driver class" # save the snapshot
```

Note

A good commit message describes *what* you accomplished, in the present tense: “Add Trip class with fare calculation” — not “stuff” or “asdf” or “final final v2”. Future you (and your instructor) will thank present you.

6.4. git push — send your commits to GitHub

A commit on your laptop is invisible to GitHub until you **push** it.

Command: git push

Purpose: Upload commits from your local branch to the same branch on GitHub.

The first time you push a new branch:

```
git push -u origin <your-name>
```

The **-u** flag tells Git to remember the link, so afterwards you can simply do:

```
git push
```

6.5. git pull — get the latest from GitHub

When the instructor releases a new lab, they push it to **master** on GitHub. To bring it into your local copy:

Command: git pull

Purpose: Fetch the latest commits from a remote branch and merge them into your current branch.

Typical use:

```
git switch master      # move to master
git pull origin master # download new lab(s) from teacher
git switch <your-name> # go back to your work branch
git merge master       # bring the new lab into your branch (
                        see next section)
```

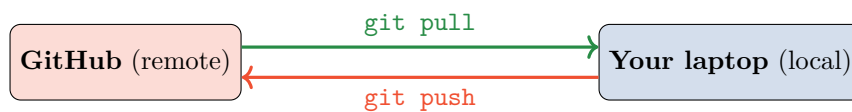


Figure 6: push sends your work up; pull brings new work down.

6.6. git merge — combine two branches

Eventually you need to combine work. Two common cases in this course:

- The teacher releases a new lab on **master** and you want it in your branch.
- You finished an assignment and the teacher wants to merge your branch into **master** for grading.

Command: git merge

Purpose: Take all the commits from one branch and integrate them into another.

Example: pull a new lab from master into your branch:

```
git switch <your-name>
git merge master
```

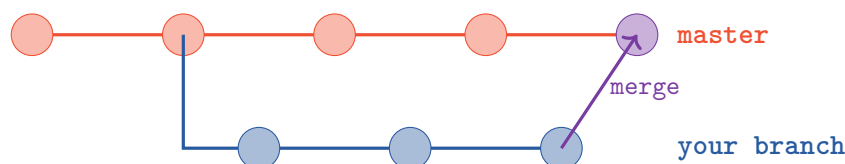


Figure 7: A merge brings the work of one branch (yours) into another (**master**).

Note

Merge conflicts happen when two branches changed the same lines of the same file. Git pauses, marks the file with «««< / ===== / >>>», and asks you to choose. You edit the file to keep what you want, then `git add` it and `git commit`. IntelliJ has an excellent visual conflict resolver — use it.

7. The Workflow for This Course

Here is the picture of how Git, GitHub, your laptop, and your instructor's laptop all fit together.

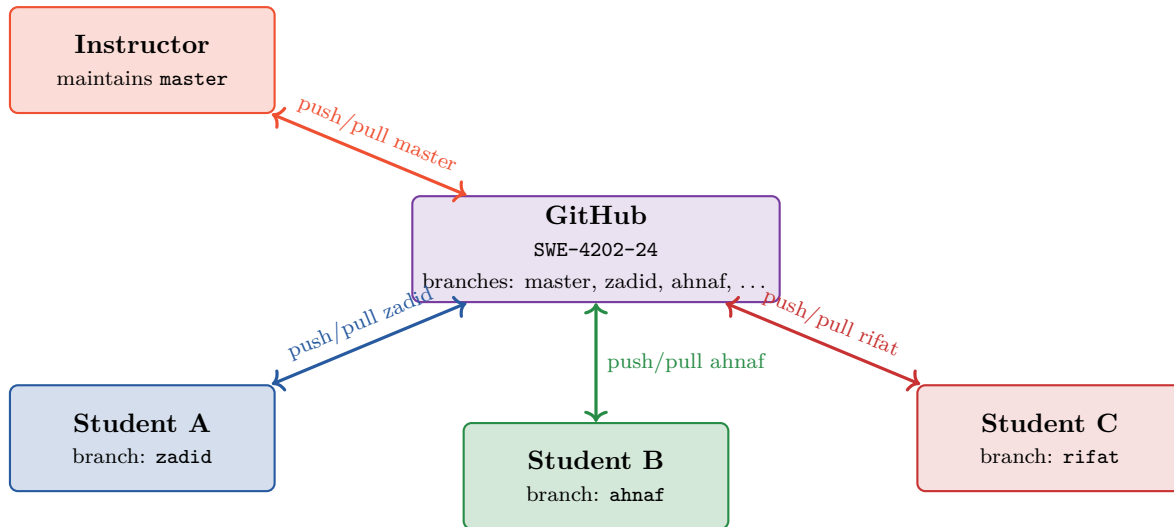


Figure 8: Everyone has their own branch. GitHub is the meeting point. The instructor's branch (**master**) is the source of truth for the official lab content.

7.1. The day-to-day rhythm

1. Once, at the start of the semester:

```
git clone <repo-url>
cd SWE-4202-24
git switch -c <your-name>
```

2. When a new lab is released:

```
git switch master
git pull origin master
git switch <your-name>
git merge master
```

3. While doing the lab: edit code in IntelliJ (with the project folder, e.g. Lab-03/RideNow, opened correctly).

4. At natural stopping points (end of a class, end of a feature, before going to bed):

```
git status
git add .
git commit -m "Meaningful message"
git push
```

5. Before submission: make sure your latest commit is on GitHub by visiting the repository page in your browser and checking that your branch shows your most recent message.

8. Common Mistakes and How to Avoid Them

Mistake 1: Opening the wrong folder in IntelliJ

You opened SWE-4202-24 or Lab-03 as the project. Symptoms: red squiggles everywhere, “no SDK” errors, `src` not recognized.

Fix: Close the project. Delete any stray `.idea/` folder created in the wrong location. Reopen Lab-03/RideNow (the folder that contains `src/`).

Mistake 2: Cloning more than once

You re-cloned the repository every time a new lab was released, ending up with SWE-4202-24, SWE-4202-24-1, SWE-4202-24-final...and your previous work scattered.

Fix: You only clone once. After that, use `git pull` to get new content.

Mistake 3: Working directly on master

You edited files while on the `master` branch, then tried to push — only to find your push was rejected because the instructor’s branch had moved on.

Fix: *Always* work on your own branch. Treat `master` as read-only; pull from it, never push to it.

Mistake 4: “It works on my laptop” but not on GitHub

You forgot to `commit` or to `push`. The code is on your machine but nobody else can see it.

Fix: After a commit, always run `git push`. Then refresh the GitHub page and verify your file is visible on your branch.

Mistake 5: Committing the `out/` or `.idea/` folders

You pushed compiled `.class` files and your personal IDE settings. The repository becomes bloated and merges conflict on machine-specific files.

Fix: Make sure each project has a `.gitignore` listing `out/`, `target/`, and `.idea/` (already done for RideNow). If you accidentally added them, run `git rm -r -cached out .idea`, then commit.

9. Quick Reference Card

Command	What it does
<code>git clone <url></code>	Download a remote repository (do this once).
<code>git status</code>	Show what's changed and what's staged.
<code>git switch <branch></code>	Move to an existing branch.
<code>git switch -c <branch></code>	Create a new branch and move to it.
<code>git add <file></code>	Stage a file for the next commit.
<code>git add .</code>	Stage all changed files.
<code>git commit -m "..."</code>	Save a snapshot with a message.
<code>git push</code>	Upload your commits to GitHub.
<code>git push -u origin
</code>	First-time push of a new branch.
<code>git pull origin
</code>	Download new commits from a branch.
<code>git merge <branch></code>	Combine another branch into the current one.
<code>git log -oneline</code>	Show a compact history of commits.

10. Closing Words

Git looks intimidating, but in this course you only need a small, repeated rhythm: *pull, work on your branch, commit, push*. The folder structure looks intimidating, but it follows one rule: *open the project folder, not the lab folder, not the repository folder*.

If you remember those two ideas, you have already solved the problems that cause ~90% of the help requests in courses like this one.